



《人工智能与Python程序设计》—— 回归分析



人工智能与Python程序设计 教研组



提纲



Python AI
Numpy与科学计算

- □ 一元线性回归
- □ 多元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 逻辑回归



一些常用名词

- 数据
 - 一般为空间中点的集合，每个点以 (x_i, y_i) 的形式出现
 - 一般称 x_i 为“特征”， y_i 为对应的“真实值”或“观测值”
- 任务：找到由 x 得到 y 的方法，即给定 x ，预测 y
- 模型：
 - 猜想 x 到 y 的映射由 $f(x)$ 给出
 - 模型即 $f(x)$ 的具体形式，带有待定参数
- 损失函数：根据已有数据，评价参数质量的指标
- 训练：找到最好的 $f(x)$ 参数的过程，即最小化损失函数的过程
- 预测：给出 x ，计算 $f(x)$ 的过程
- 测试：评价预测结果



优化：梯度下降法

- 1. 随机选择一组初始参数(w^0, b^0)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$w^{t+1} = w^t - \boxed{lr} \frac{\partial L(w, b)}{\partial w} (w^t)$$

学习率

$$b^{t+1} = b^t - \boxed{lr} \frac{\partial L(w, b)}{\partial b} (b^t)$$

- 4. 循环迭代步骤 3，直到前后两次迭代得到的(w^t, b^t)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出(w^t, b^t)，即为使得损失函数最小时的参数取值。

示例：梯度下降法

```
class LinearRegression(object):  
  
    def __init__(self, learning_rate=0.01, max_iter=100, seed=None):  
        """  
        一元线性回归类的构造函数：  
        参数 学习率：learning_rate  
        参数 最大迭代次数：max_iter  
        参数 seed：产生随机数的种子  
        从正态分布中采样w和b的初始值  
        """  
        np.random.seed(seed)  
        self.lr = learning_rate  
        self.max_iter = max_iter  
        self.w = np.random.normal(1, 0.1)  
        self.b = np.random.normal(1, 0.1)  
        self.loss_arr = []
```



示例：梯度下降法

```
def fit(self, x, y):
```

类的方法：训练函数

参数 自变量：x

参数 因变量：y

返回每一次迭代后的损失函数

'''

```
for i in range(self.max_iter):
```

```
    self.__train_step(x, y)
```

```
    y_pred = self.predict(x)
```

```
    self.loss_arr.append(self.loss(y, y_pred))
```

```
def __f(self, x, w, b):
```

'''

类的方法：计算一元线性回归函数在x处的值

'''

```
return x * w + b
```

```
def predict(self, x):
```

'''

类的方法：预测函数

参数：自变量：x

返回：对x的回归值

'''

```
y_pred = self.__f(x, self.w, self.b)
```

```
return y_pred
```

```
def loss(self, y_true, y_pred):
```

'''

类的方法：计算损失

参数 真实因变量：y_true

参数 预测因变量：y_pred

返回：MSE损失

'''

```
return np.mean((y_true - y_pred)**2)
```



示例：梯度下降法

```
def __calc_gradient(self, x, y):
```

类的方法：分别计算对w和b的梯度

```
    d_w = np.mean(2*(x * self.w + self.b - y) * x)
    d_b = np.mean(2*(x * self.w + self.b - y))
    return d_w, d_b
```

```
def __train_step(self, x, y):
```

类的方法：单步迭代，即一次迭代中对梯度进行更新

```
    d_w, d_b = self.__calc_gradient(x, y)
    self.w = self.w - self.lr * d_w
    self.b = self.b - self.lr * d_b
    return self.w, self.b
```



示例：梯度下降法

```
In [36]: import numpy as np
import matplotlib.pyplot as plt

def show_data(x, y, w=None, b=None):
    plt.scatter(x, y, marker='.')
    if w is not None and b is not None:
        plt.plot(x, w*x+b, c='red')
    plt.show()

# data generation
np.random.seed(272)
data_size = 100
x = np.random.uniform(low=1.0, high=10.0, size=data_size)
y = x * 20 + 10 + np.random.normal(loc=0.0, scale=10.0, size=data_size)
```




示例：梯度下降法

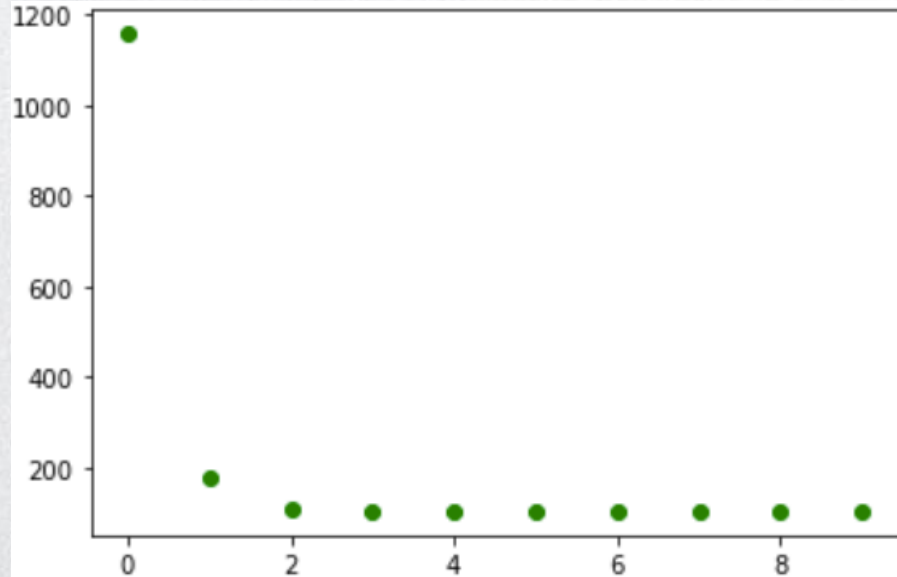
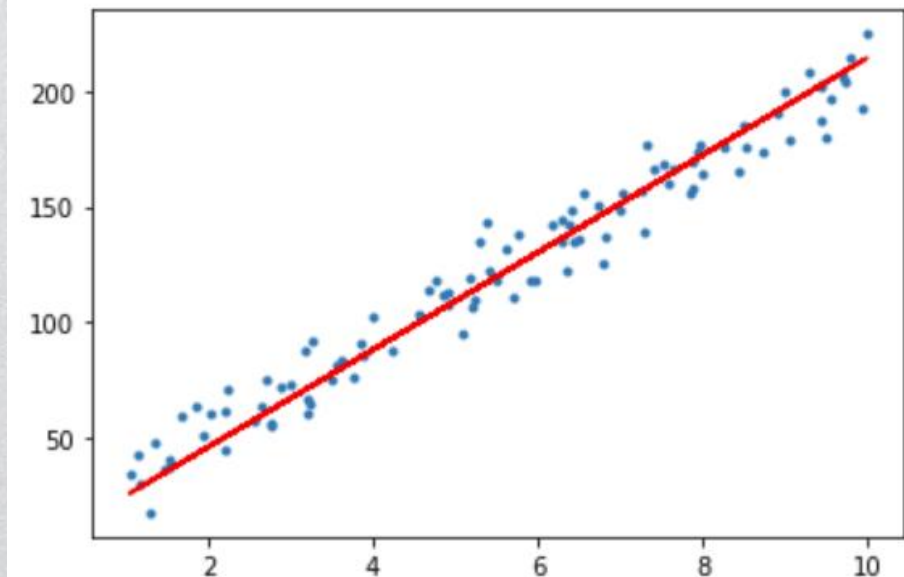
```
# train / test split
{ shuffled_index = np.random.permutation(data_size)
  x = x[shuffled_index]
  y = y[shuffled_index]
  split_index = int(data_size * 0.7)
  x_train = x[:split_index]
  y_train = y[:split_index]
  x_test = x[split_index:]
  y_test = y[split_index:]

# train the liner regression model
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)
print('w: \t{:.3}'.format(regr.w))
print('b: \t{:.3}'.format(regr.b))
show_data(x, y, regr.w, regr.b)

# plot the evolution of cost
plt.scatter(np.arange(len(regr.loss_arr)), regr.loss_arr, marker='o', c='green')
plt.show()
```

示例：梯度下降法

w: 21.0
b: 4.41



思考



- 多元线性回归：
 - 自变量 x 是向量，因变量 y 是数字
 - 自变量 x 是向量，因变量 y 也是向量
- 怎样用梯度下降法训练多元线性回归模型？



提纲

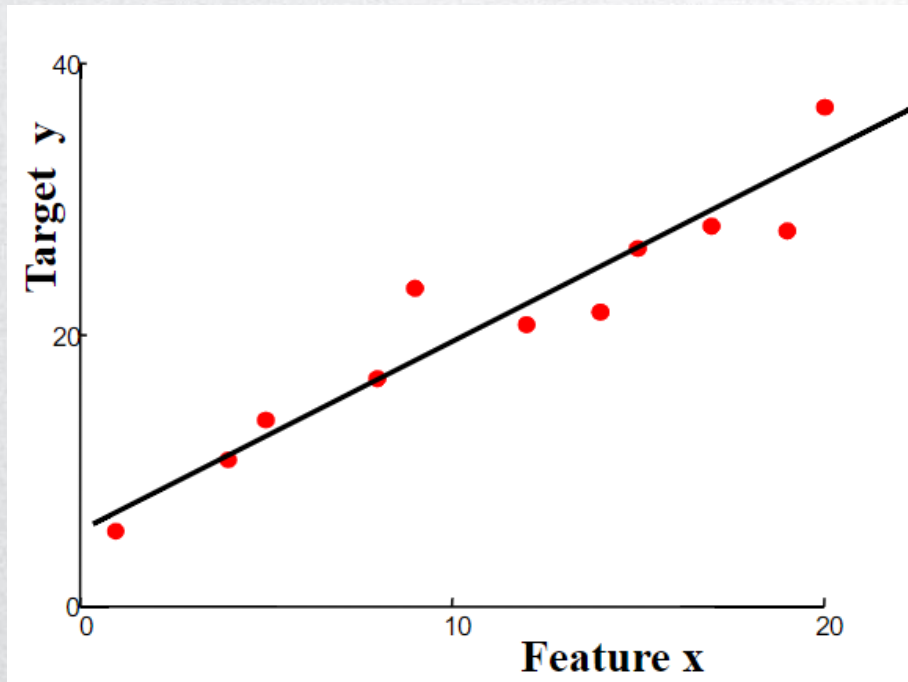


Python AI
Numpy与科学计算

- □ 一元线性回归
- □ 多元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 逻辑回归

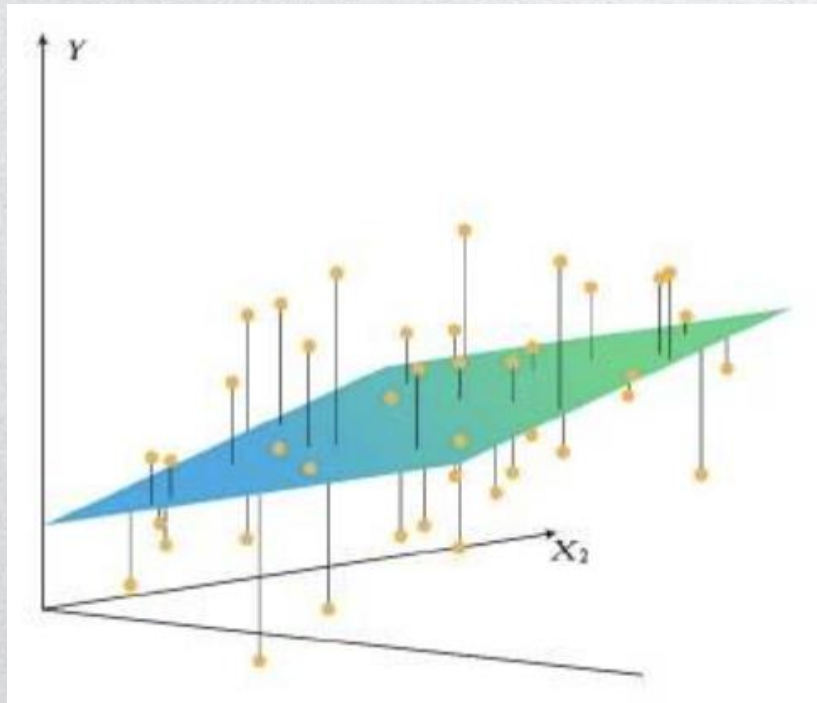
一元线性回归

- 只有一个自变量（特征 x 的维度为1，因此 w 的维度也是1）



多元线性回归

- 有多个自变量 (特征 x 的维度大于1, 对应的 w 的维度也大于1)



$$\mathbf{x}_i = \begin{pmatrix} x_{i,1} \\ x_{i,2} \\ \dots \\ x_{i,j} \\ \dots \\ x_{i,d} \end{pmatrix}$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_j \\ \dots \\ w_d \end{pmatrix}$$



多元线性回归

- 假设目标值与特征之间线性相关，即因变量与自变量满足一个多元一次方程。
- 训练集 $D=\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- 将特征 $\mathbf{x}$ 和对应的目标 y 建模为多元一次方程：

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

其中， $\mathbf{w}$ 和 b 为模型的参数。

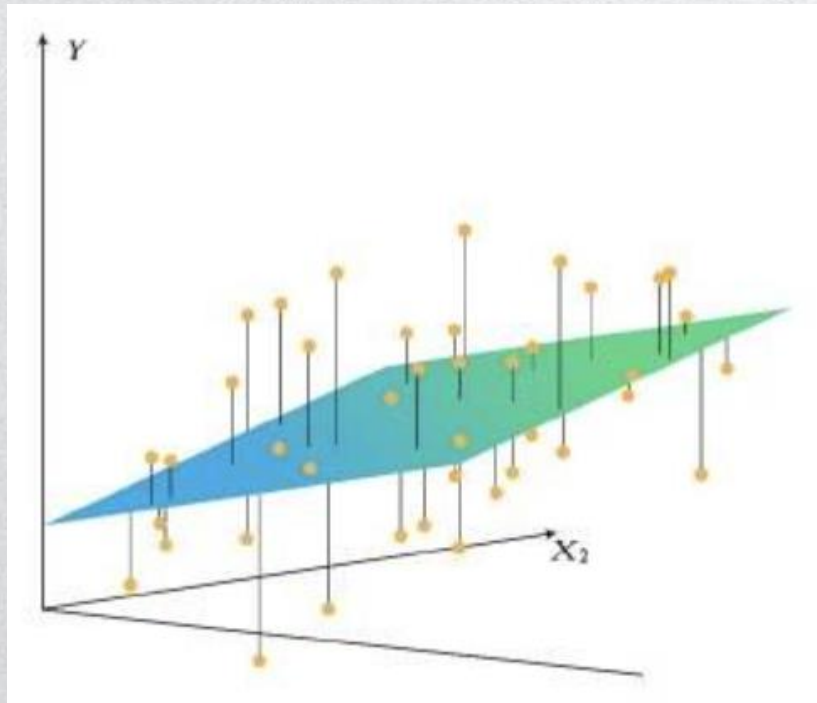
- 为了简化符号表示，在 $\mathbf{x}$ 后面加上一维值恒为1的特征， $\mathbf{x} \leftarrow [\mathbf{x}, 1]$ ，同时 $\mathbf{w}$ 增加一维，则可以把 b 融入 $\mathbf{w}$

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

- 为了学习这两个参数，根据已知的训练样本点（自变量 $\mathbf{x}$ 和因变量 y 都是已知的）拟合这个多元一次方程，求解参数。

多元线性回归

- 有多个自变量（特征x的维度大于1，对应的w的维度也大于1）



$$\mathbf{x}_i = \begin{pmatrix} x_{i,1} \\ x_{i,2} \\ \dots \\ x_{i,j} \\ \dots \\ x_{i,d} \\ 1 \end{pmatrix}$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_j \\ \dots \\ w_d \\ w_{d+1} \end{pmatrix}$$

损失函数

- 将线性预测函数式代入MSE损失函数 $L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$ ，将需要求解的参数 w 看做是损失函数 L 的自变量：

$$L(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$$

- 通过最小化损失函数求解最优参数：

$$(w^*) = \arg \min_w \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$$

或者 $(w^*) = \arg \min_w \frac{1}{N} \sum_{i=1}^N (x_i^T w - y_i)^2$

求梯度

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

- 分别对参数 $\mathbf{w}$ 求导:

$$\frac{\partial L}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N x_{i,1} (\mathbf{x}_i^T \mathbf{w} - y_i)$$

$$\frac{\partial L}{\partial w_2} = \frac{2}{N} \sum_{i=1}^N x_{i,2} (\mathbf{x}_i^T \mathbf{w} - y_i)$$

.....

$$\frac{\partial L}{\partial w_{d+1}} = \frac{2}{N} \sum_{i=1}^N x_{i,d+1} (\mathbf{x}_i^T \mathbf{w} - y_i)$$

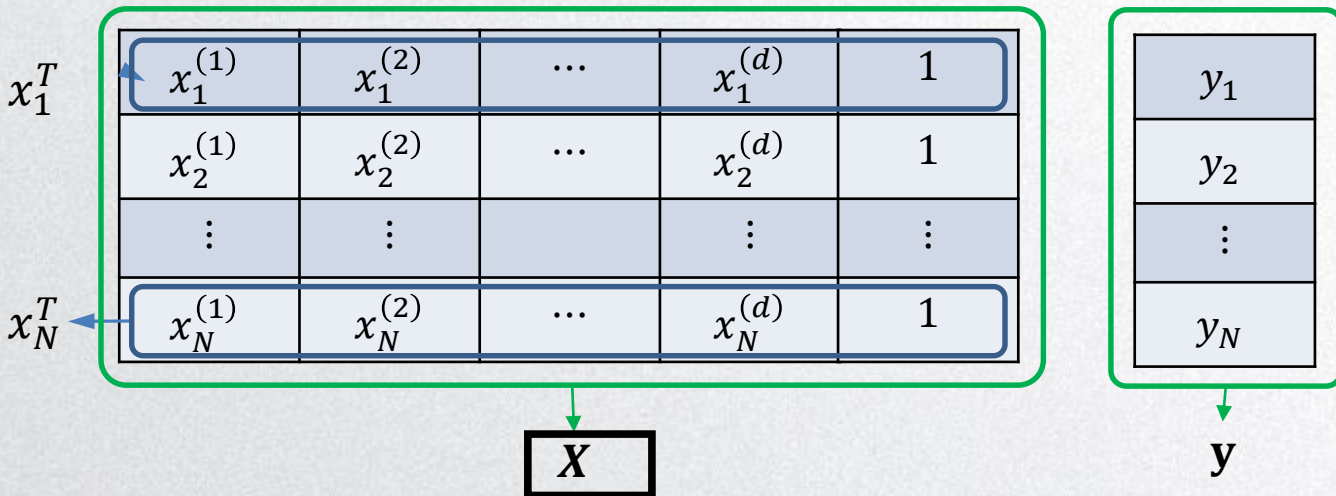
求梯度

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

- 分别对参数 $\mathbf{w}$ 求导：

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \begin{pmatrix} \frac{2}{N} \sum_{i=1}^N x_{i,1} (\mathbf{x}_i^T \mathbf{w} - y_i) \\ \frac{2}{N} \sum_{i=1}^N x_{i,2} (\mathbf{x}_i^T \mathbf{w} - y_i) \\ \dots \\ \frac{2}{N} \sum_{i=1}^N x_{i,d+1} (\mathbf{x}_i^T \mathbf{w} - y_i) \end{pmatrix} = \frac{2}{N} \sum_{i=1}^N \mathbf{x}_i (\mathbf{x}_i^T \mathbf{w} - y_i)$$

梯度的矩阵形式



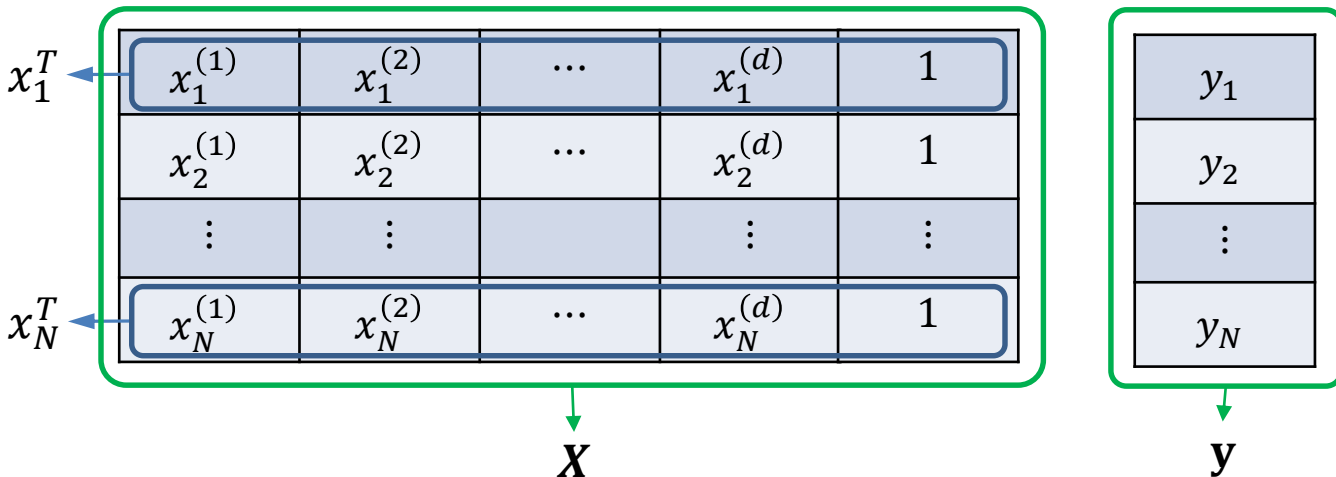
$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N x_i (x_i^T \mathbf{w} - y_i), \text{ 其中 } x_i \text{ 为列向量}$$

- 矩阵形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d + 1)$, $\mathbf{w}$ 维度为 $(d + 1) \times 1$

梯度的矩阵形式



$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N x_i (x_i^T \mathbf{w} - y_i), \text{ 其中 } x_i \text{ 为列向量}$$

- 矩阵形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d + 1)$, $\mathbf{w}$ 维度为 $(d + 1) \times 1$



优化：梯度下降法

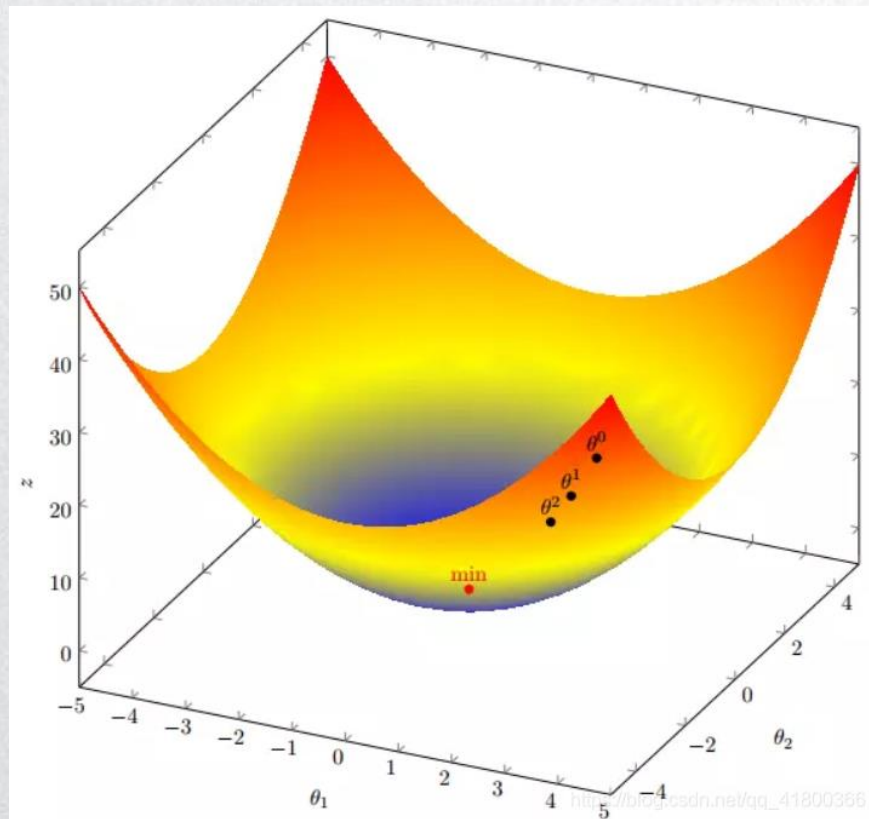
- 1. 随机选择一组初始参数($\mathbf{w}^0$)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} (\mathbf{w}^t)$$

- 4. 循环迭代步骤 3，直到前后两次迭代得到的($\mathbf{w}^t$)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出($\mathbf{w}^t$)，即为使得损失函数最小时的参数取值。



梯度下降法





示例：梯度下降法（类的定义和初始化）

```
class MultiLinearRegression(object):  
    def __init__(self, dim_in, learning_rate=0.01, max_iter=100, seed=None):  
        """  
        一元线性回归类的构造函数：  
        参数 学习率: learning_rate  
        参数 最大迭代次数: max_iter  
        参数 seed: 产生随机数的种子  
        从正态分布中采样w的初始值  
        """  
        np.random.seed(seed)  
        self.lr = learning_rate  
        self.max_iter = max_iter  
        self.w = np.random.normal(1, 0.1, [dim_in+1, 1]) # w 的维度为输入维度+1  
        self.loss_arr = []
```




示例：梯度下降法

```
def fit(self, x, y):
```

```
    """
```

```
    类的方法：训练函数
```

```
    参数 自变量: x
```

```
    参数 因变量: y
```

```
    返回每一次迭代后的损失函数
```

```
    """
```

```
    # 首先在x矩阵后面增加一列1
```

```
    x = np.hstack([x, np.ones((x.shape[0], 1))])
```

```
    for i in range(self.max_iter):
```

```
        self.__train_step(x, y)
```

```
        y_pred = self.predict(x)
```

```
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __train_step(self, x, y):
```

```
    """
```

```
    类的方法：单步迭代，即一次迭代中对梯度进行更新
```

```
    """
```

```
    d_w = self.__calc_gradient(x, y)
```

```
    self.w = self.w - self.lr * d_w
```

```
    return self.w
```

```
def loss(self, y_true, y_pred):
```

```
    """
```

```
    类的方法：计算损失
```

```
    参数 真实因变量: y_true
```

```
    参数 预测因变量: y_pred
```

```
    返回: MSE 损失
```

```
    """
```

```
    return np.mean((y_true - y_pred) ** 2)
```

```
def __calc_gradient(self, X, y):
```

```
    """
```

```
    类的方法：分别计算对w和b的梯度
```

```
    """
```

```
    N = X.shape[0]
```

```
    #diff = (X.dot(self.w) - y)
```

```
    diff = np.matmul(X, self.w) - y
```

```
    #print(type(X.T))
```

```
    grad = np.matmul(X.T, diff) # X.T 转置 x.transpose()
```

```
    d_w = (2 * grad) / N
```

```
    return d_w
```

```
def __f(self, X, w):
```

```
    """
```

```
    类的方法：计算一元线性回归函数在x处的值
```

```
    """
```

```
    return np.matmul(X, w)
```

```
def predict(self, X):
```

```
    """
```

```
    类的方法：预测函数
```

```
    参数：自变量: x
```

```
    返回：对x的回归值
```

```
    """
```

```
    y_prd = self.__f(X, self.w)
```

```
    return y_prd
```



示例：梯度下降法（数据准备）

29

```
# data generation
np.random.seed(272)
data_size = 100
dim_in = 3
dim_out = 1
x = np.random.uniform(low=1.0, high=10.0, size=[data_size, dim_in])
map_true = np.array([[1.5], [-5.], [3.]])
y = x.dot(map_true) + np.random.normal(loc=0.0, scale=10.0, size=[data_size, dim_out])

# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index, :]
y = y[shuffled_index, :]
split_index = int(data_size * 0.7)
x_train = x[:split_index, :]
y_train = y[:split_index, :]
x_test = x[split_index:, :]
y_test = y[split_index:, :]
```



示例：梯度下降法（训练模型）

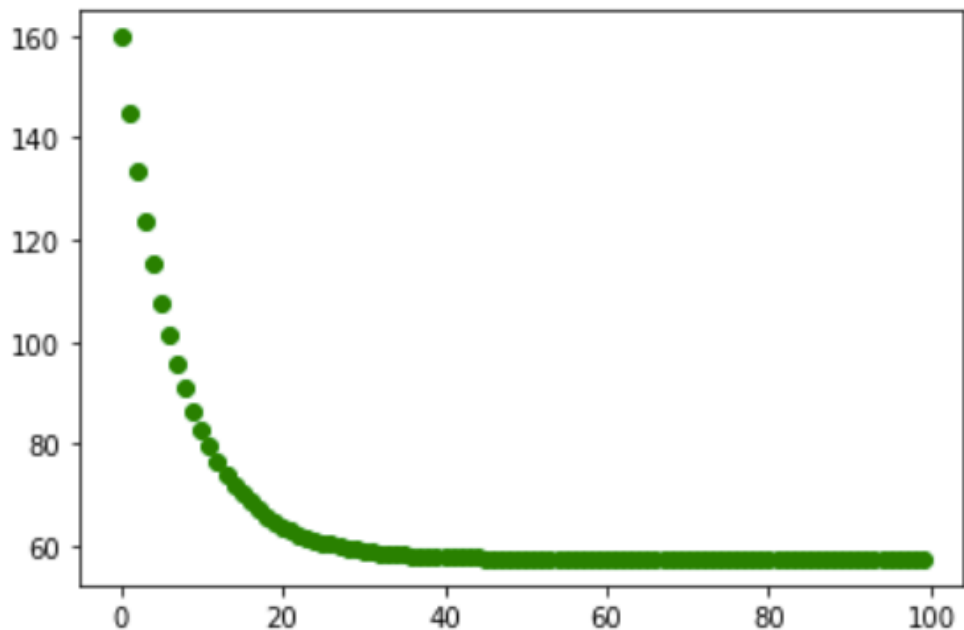
```
# train the liner regression model
regr = LinearRegressionMulti(dim_in, learning_rate=0.01, max_iter=100, seed=0)
regr.fit(x_train, y_train)
print(regr.w)

x_test_aug = np.hstack([x_test, np.ones((x_test.shape[0], 1))])
y_pred = regr.predict(x_test_aug)
res = y_pred - y_test

# plot the evolution of cost
plt.scatter(np.arange(len(regr.loss_arr)), regr.loss_arr, marker='o', c='green')
plt.show()
```

示例：梯度下降法

```
[[ 0.53989083]  
 [-4.89837128]  
 [ 3.30861944]  
 [ 0.78811499]]
```





提纲



Python AI
Numpy与科学计算

- □ 一元线性回归
- □ 多元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 逻辑回归



matplotlib库简介

- Matplotlib是一个功能强大的Python绘图和可视化库
 - 我们主要介绍其中pyplot子库的功能
- 安装方式
 - `pip install matplotlib`
- 官方主页：
 - <https://matplotlib.org/>

matplotlib

使用Matplotlib库进行图表绘制

- 绘图环境设置:

```
[58]: # 导入pyplot子库
import matplotlib.pyplot as plt

# 对绘图环境进行设置, 选择使用中文字体
import matplotlib
matplotlib.rcParams['font.family']='SimHei'
matplotlib.rcParams['font.sans-serif']=['SimHei']

# 使jupyter notebook能直接显示matplotlib绘制的结果
%matplotlib inline
```



使用Matplotlib库进行图表绘制

- 以一元线性回归和逻辑回归问题为例，介绍如何使用matplotlib绘制：
 - 散点图
 - 折线图
 - 直方图
 - 二维直方图
 - 等高线图



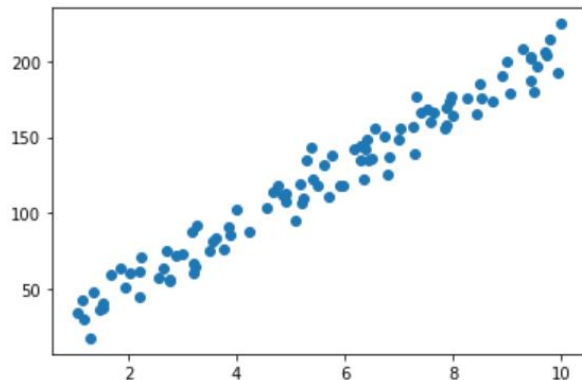
使用Matplotlib库进行图表绘制

• 散点图

- 展示(x, y)数据点的分布，使用plt.scatter(x, y)函数绘制
 - x, y是两个长度相等的list，或shape为(n,)的一维ndarray，表示点的坐标
 - 可选参数：
 - maker: 点的样式（参见: https://matplotlib.org/stable/api/markers_api.html#module-matplotlib.markers）
 - c: 点的颜色
- 以一元线性回归问题中生成的数据为例：

```
[5]: import numpy as np

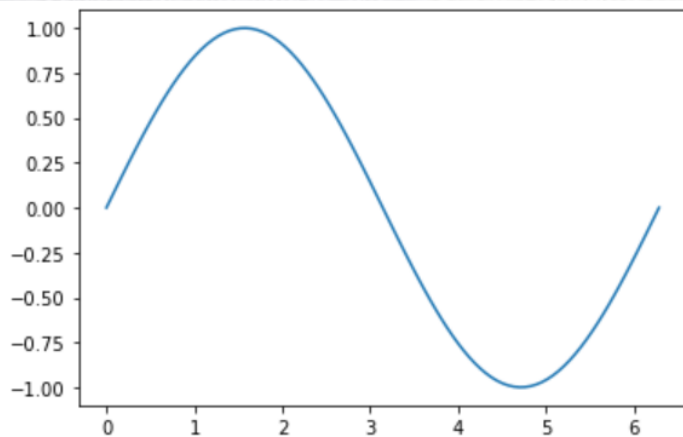
# 生成数据
np.random.seed(272)
data_size = 100
# x在1到10间均匀分布
x = np.random.uniform(low=1.0, high=10.0, size=data_size)
# y = x * 20 + 10 + 随机噪声
y = x * 20 + 10 + np.random.normal(loc=0.0, scale=10.0, size=data_size)
plt.scatter(x, y)
plt.show()
```



使用Matplotlib库进行图表绘制

- 折线图：
 - 展示y随x的变化情况，使用`plt.plot(x, y, [fmt])`函数绘制
 - x, y是两个长度相等的list，或shape为(n,)的一维ndarray，表示点的坐标
 - 同样可以用额外的可选参数 (fmt) 控制绘制的样式
 - 请在jupyter notebook下执行`plt.plot?`查看相关帮助
 - 例：绘制Sin函数在 $[0, 2\pi]$ 范围内的图像

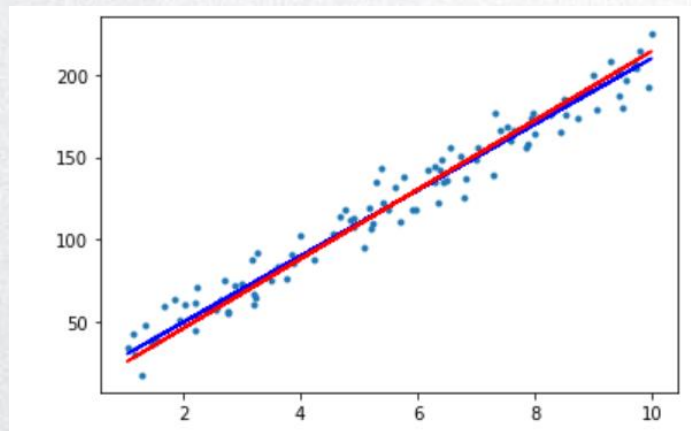
```
[12]: # 绘制折线图
x = np.linspace(0, 2*np.pi, num=100)
y = np.sin(x)
plt.plot(x, y)
plt.show()
```



使用Matplotlib库进行图表绘制

- 折线图：
 - 展示y随x的变化情况，使用plt.plot(x, y)函数绘制
 - 例：展示一元线性回归模型在训练集上学习到的直线方程
 - $y = f(x) = wx + b$
 - 并与数据点及生成数据的数据方程进行比较
 - 在执行plt.show()之前执行多次绘图函数

```
[20]: # 绘制数据点
plt.scatter(x, y, marker='.')
# 用蓝线绘制生成数据时的方程
plt.plot(x, x * 20 + 10, 'b')
# 用红线绘制在训练集上拟合得到的直线方程
plt.plot(x, x * regr.w + regr.b, 'r')
plt.show()
```



使用Matplotlib库进行图表绘制

- 直方图：
 - 展示一维数据的分布，使用`plt.hist(x)`函数绘制
 - 将数据按照大小放到n个桶里，用柱状图展示每个桶里的数据点个数
 - x: 包含需要展示的数据的list或者一维ndarray
 - 部分可选参数：
 - bins: 桶的个数 (或者以序列形式给出的桶的边界)
 - color: 直方图的颜色

使用Matplotlib库进行图表绘制

• 直方图:

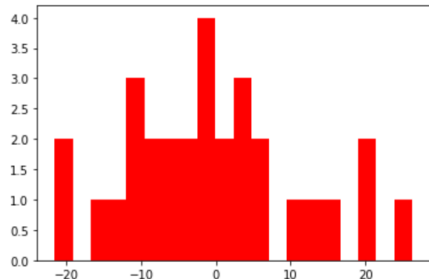
– 例：展示一元线性回归模型的测试误差的分布

$$\bullet \text{error}_i = \hat{y}_i - y_i = wx_i + b - y_i$$

```
[68]: # 生成数据
# 数据集大小为100
x, y, x_train, y_train, x_test, y_test = generate_data(100)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

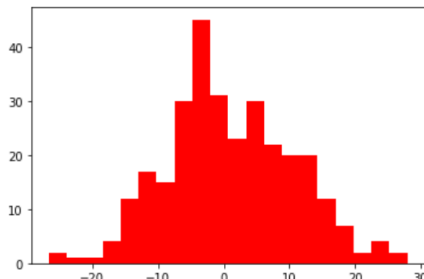
# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



```
[82]: # 生成数据
# 数据集大小为1000
x, y, x_train, y_train, x_test, y_test = generate_data(1000)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

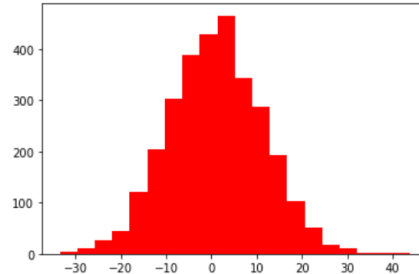
# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



```
[83]: # 生成数据
# 数据集大小为10000
x, y, x_train, y_train, x_test, y_test = generate_data(10000)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



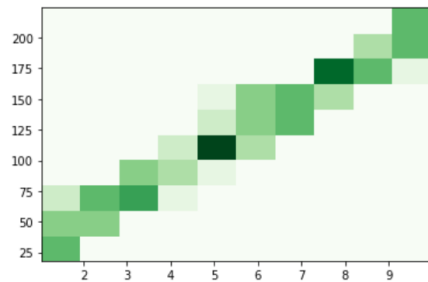
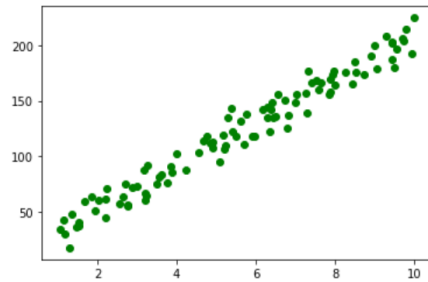
使用Matplotlib库进行图表绘制

- 二维直方图（热度图）：
 - 展示**二维**数据的分布，使用`plt.hist2d(x, y)`函数绘制
 - 将数据按照(x, y)坐标放到 $n*n$ 个桶里，用**颜色**每个桶里的数据点个数
 - x, y: 两个长度相等的list, 或shape为(n,)的一维ndarray, 表示点的坐标
 - 可选参数：
 - cmap: 使用不同的颜色来绘制分布

```
[93]: # 生成数据
x, y, x_train, y_train, x_test, y_test = generate_data(100, noise_scale=10.0)

# 散点图
plt.scatter(x, y, c='g')
plt.show()

# 二维直方图
plt.hist2d(x, y, cmap='Greens')
plt.show()
```



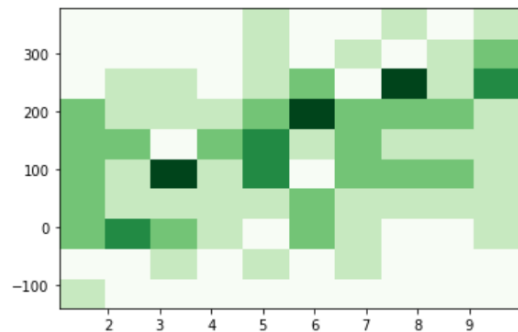
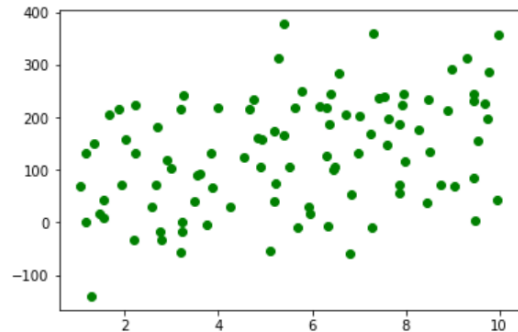
使用Matplotlib库进行图表绘制

- 二维直方图（热度图）：
 - 展示**二维**数据的分布，使用plt.hist2d(x, y)函数绘制
 - 改变数据的噪声

```
[94]: # 生成数据
x, y, x_train, y_train, x_test, y_test = generate_data(100, noise_scale=100.0)

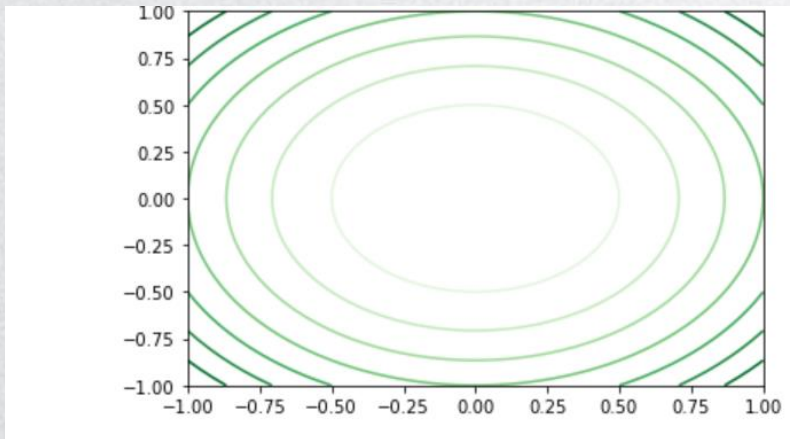
# 散点图
plt.scatter(x, y, c='g')
plt.show()

# 二维直方图
plt.hist2d(x, y, cmap='Greens')
plt.show()
```



使用Matplotlib库进行图表绘制

- 等高线图：
 - 展示三维数据的分布，使用`plt.contour()`函数绘制
 - 用等高线的方式，绘制出 $z = f(x, y)$ 的图像
 - 例：展示 $z = x^2 + y^2$



使用Matplotlib库进行图表绘制

- 等高线图:
 - 例：展示 $z = x^2 + y^2$
 - 第一步：生成 (x, y) 坐标网格

```
[113]: # 例：绘制 $z=x^2+y^2$   
# 给定 $x$ 和 $y$ 的范围  
x = np.linspace(-1, 1, num=5)  
y = np.linspace(-1, 1, num=5)  
# 使用np.meshgrid函数生成一个网格  
x, y = np.meshgrid(x, y)  
print(x)  
print(y)
```

```
[[ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]]  
[[ -1.  -1.  -1.  -1.  -1. ]  
 [-0.5 -0.5 -0.5 -0.5 -0.5]  
 [ 0.   0.   0.   0.   0. ]  
 [ 0.5  0.5  0.5  0.5  0.5]  
 [ 1.   1.   1.   1.   1. ]]
```

使用Matplotlib库进行图表绘制

- 等高线图:

- 例: 展示 $z = x^2 + y^2$

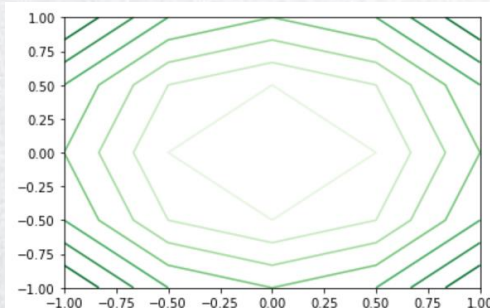
- 第二步: 计算 z 在 (x, y) 坐标网格上每一点的值

```
[115]: # 计算z在网格上每一点的值
z = x**2 + y**2
print(z)

[[2.   1.25 1.   1.25 2.   ]
 [1.25 0.5  0.25 0.5  1.25]
 [1.   0.25 0.   0.25 1.   ]
 [1.25 0.5  0.25 0.5  1.25]
 [2.   1.25 1.   1.25 2.   ]]
```

- 第三步: 绘制等高线图
 - 同样可以用cmap调整颜色

```
[117]: plt.contour(x, y, z, cmap="Greens")
plt.show()
```



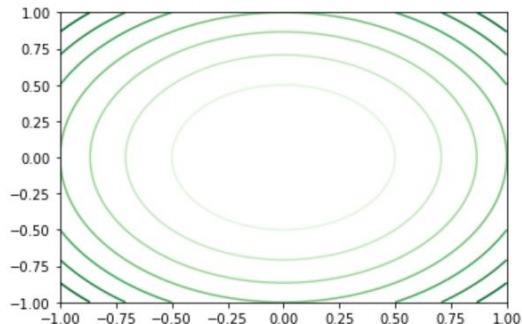
使用Matplotlib库进行图表绘制

- 等高线图:

- 例: 展示 $z = x^2 + y^2$

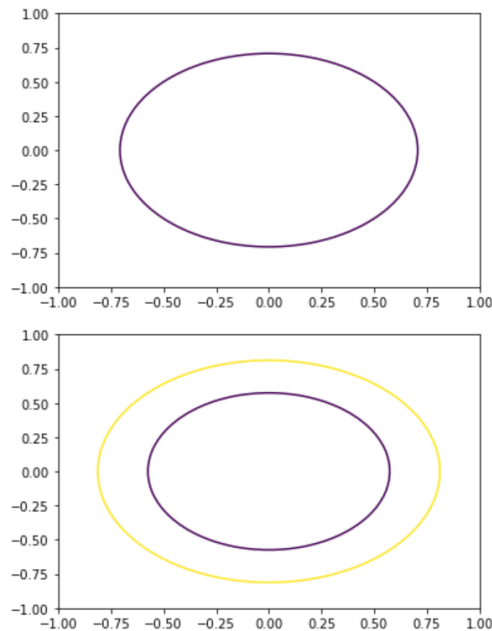
- 使用更密的网格, 使图像更平滑
 - 使用可选参数
 - 制定绘制所需的等高线

```
[119]: # 用更多的点, 使函数更为平滑
x = np.linspace(-1, 1, num=50)
y = np.linspace(-1, 1, num=50)
x, y = np.meshgrid(x, y)
z = x**2 + y**2
plt.contour(x, y, z, cmap="Greens")
plt.show()
```



```
[123]: # 只保留z=0.5一条等高线
plt.contour(x, y, z, [0.5])
plt.show()

# 保留z=[0.33, 0.66]两条等高线
plt.contour(x, y, z, [0.33, 0.66])
plt.show()
```



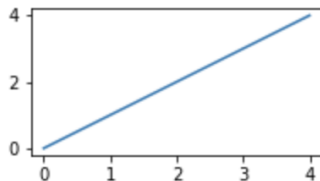
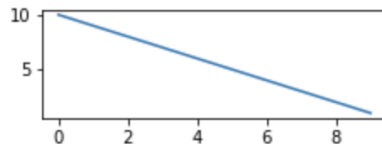
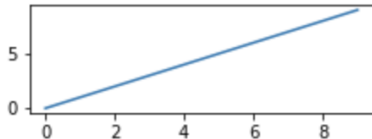
使用Matplotlib库进行图表绘制

- 绘制多个子图:

```
# 绘图环境的设置
plt.figure(figsize=(8,4)) # 设置画布大小
plt.subplot(321) # 在一个三行两列的绘图区域中选择子区域1开始绘图
plt.plot(range(10), range(10)) # x=0..9, y=0..9

plt.subplot(324) # 在一个三行两列的绘图区域中选择子区域4开始绘图
plt.plot(range(10), range(10,-1))

# 直接在画布上按照[左边缘位置, 下边缘位置, 宽度, 高度]选择绘图区域
plt.axes([0.1, 0.1, 0.3, 0.3])
plt.plot(range(5), range(5))
plt.show()
```





使用Matplotlib库进行图表绘制

- 保存绘制的图像：
 - 将plt.show()换成plt.savefig(文件名)
 - 用文件名中的扩展名 (.xxx) 指定保存的图片文件的格式
 - 支持的格式：
 - 位图：
 - 由二维排布的像素组成的图像
 - 像素放大后会有明显的锯齿
 - JPG (有损压缩)
 - PNG (无损)
 - 矢量图：
 - 用点、直线或者多边形等基于数学方程的几何图元表示图像
 - 可平滑放大
 - SVG
 - PDF



小结

- matplotlib是一个功能强大的python绘图库
- 主要学习了基于pyplot子库绘制不同类型的图表
- 可以将绘制好的图表保存成位图或者矢量图



提纲



Python AI
Numpy与科学计算

- □ 一元线性回归
- □ 多元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 逻辑回归

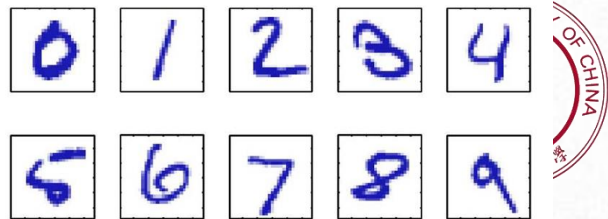


回归与分类

- 都是监督学习中常见的任务类型

- 区别

- 任务类型不同
- 输出值不同
 - 回归模型一般给出**具体数值**，**输出值连续**
 - 分类模型一般给出**所属类别**，**输出值离散**
- 使用的模型不同



Images are 28 x 28 pixels

输入：每一个图像表示为28*28维的向量 $\mathbf{x} \in R^{784}$
学习一个分类器 $f: \mathbf{x} \mapsto \{0,1,2,3,4,5,6,7,8,9\}$

标签	短信内容
0	商业秘密的秘密性那是维系其商业价值和垄断地位的前提条件之一
1	南口阿玛施新春第一批限量春装到店啦 春暖花开淑女裙、冰蓝色公主衫 气质粉小西装、冰丝女王长半裙
0	带给我们大常州一场壮观的视觉盛宴
0	有原因不明的泌尿系统结石等
0	23年从盐城拉回来的麻麻的嫁妆
1	感谢致电杭州萧山全金釜韩国烧烤店，本店位于金城路xxx号。韩式烧烤等，价格实惠、欢迎惠顾【全金釜韩国烧烤店】
1	(长期诚信在本市作各类资格职称（以及印/章、牌、.....等。祥：xxxxxxxxxxxx李伟%）

输入：每一个短信文本
学习一个分类器 $f: \mathbf{x} \mapsto \{\text{正常短信}, \text{垃圾短信}\}$

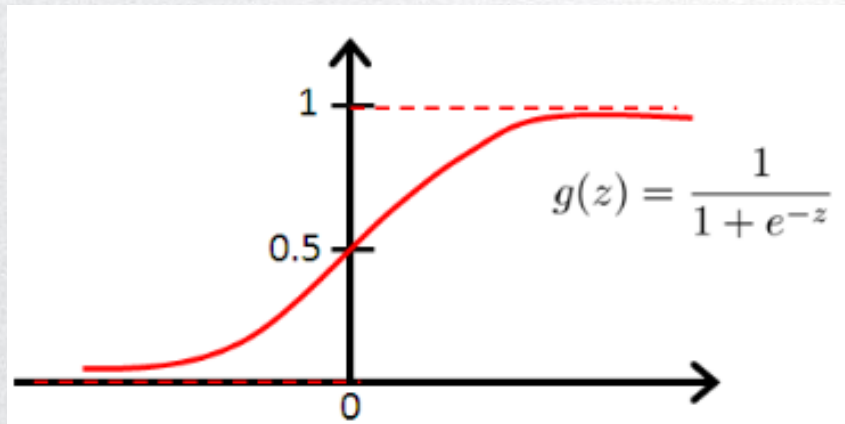
二分类问题与Logistic回归

- 二分类：因变量 y 可能属于的两个类分别称为负类和正类，则因变量 $y \in \{0, 1\}$ ，其中0表示负类，1表示正类。
- 训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$
- Logistic回归：引入非线性变换sigmoid函数 $g(z)$ ，把线性回归的输出值压缩到 $(0, 1)$ 之间：

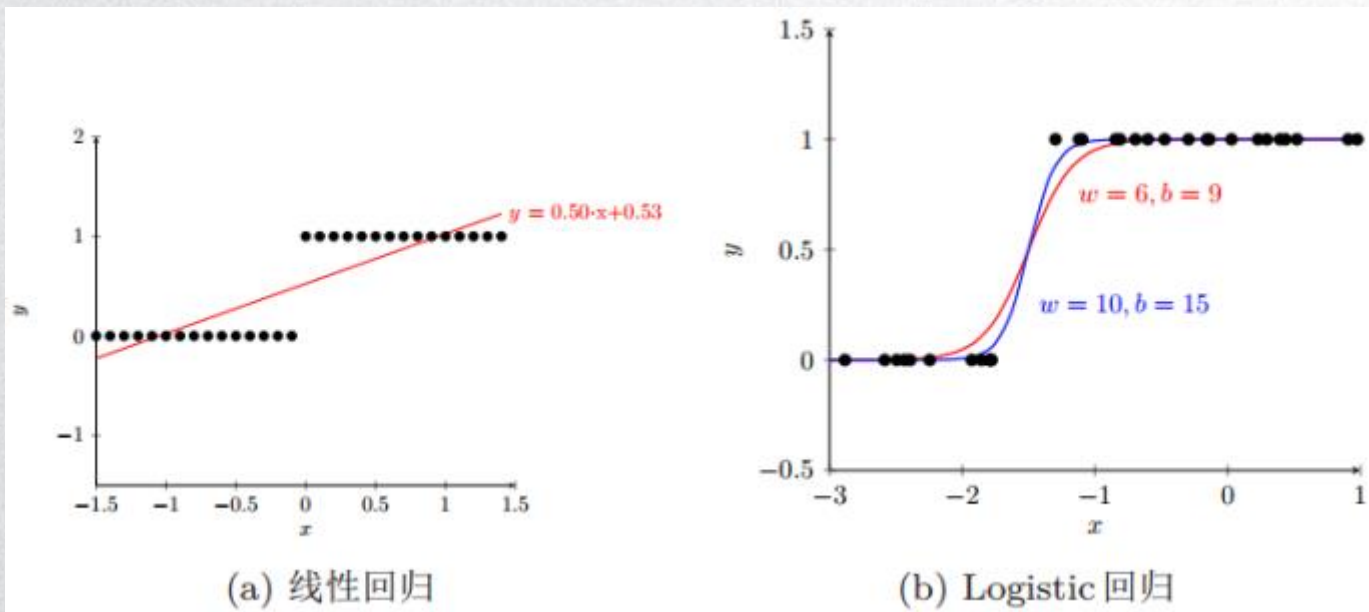
$$\hat{y} = g(w^T x)$$

其中， w 为模型的参数。

注意：已经在 x 后添加恒为1的特征，
将参数 b 融入了 w



线性回归与逻辑回归





逻辑回归

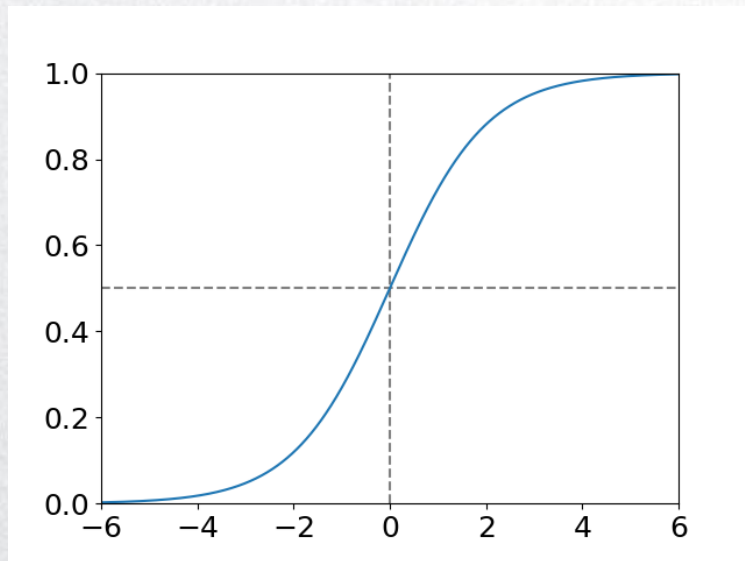
- 什么是“回归”？
 - 预测的是落入某一分类的概率
- 又叫“对数几率”回归
 - 设某事件发生概率为 $p \in [0,1]$
 - 若采用线性回归，则无法保证输出在 $[0,1]$ 范围内
 - 机率比（发生比、优势比）： $\frac{p}{1-p} \geq 0$
 - 取对数，令 $z = \ln \frac{p}{1-p} \in (-\infty, +\infty)$
 - 可以采用线性回归模型拟合： $z = f(x) = \mathbf{w}^T \mathbf{x}$

逻辑回归

- 逻辑回归模型:

$$\mathbf{w}^T \mathbf{x} = z = \ln \frac{p}{1-p} \Rightarrow p = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-(\mathbf{w}^T \mathbf{x})}}$$

- Sigmoid函数 $p = \frac{1}{1+e^{-z}} \in (0,1)$
 - 一般用于表示概率



逻辑回归损失函数的构造

- 什么样的参数 $\mathbf{w}$ 会使得观察到训练数据 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ 的概率最大?
 - 对于单个训练数据 $(\mathbf{x}_i, y_i)$
 - 如果为正例: $y_i = 1$, 使得 p_i 最大
 - 如果为负例: $y_i = 0$, 使得 $1 - p_i$ 最大

↓

 - 总体而言: 使得 $p_i^{y_i} \times (1 - p_i)^{1-y_i}$ 最大
 - 取对数: 使得 $y_i \ln p_i + (1 - y_i) \ln(1 - p_i)$ 最大
- 对于 N 个训练数据 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$, 使得它们的联合概率越大 (负联合概率最小)

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$



逻辑回归

- 损失函数

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

- 一般称为交叉熵损失函数
- 应用梯度下降法：梯度 (j 是维度索引)

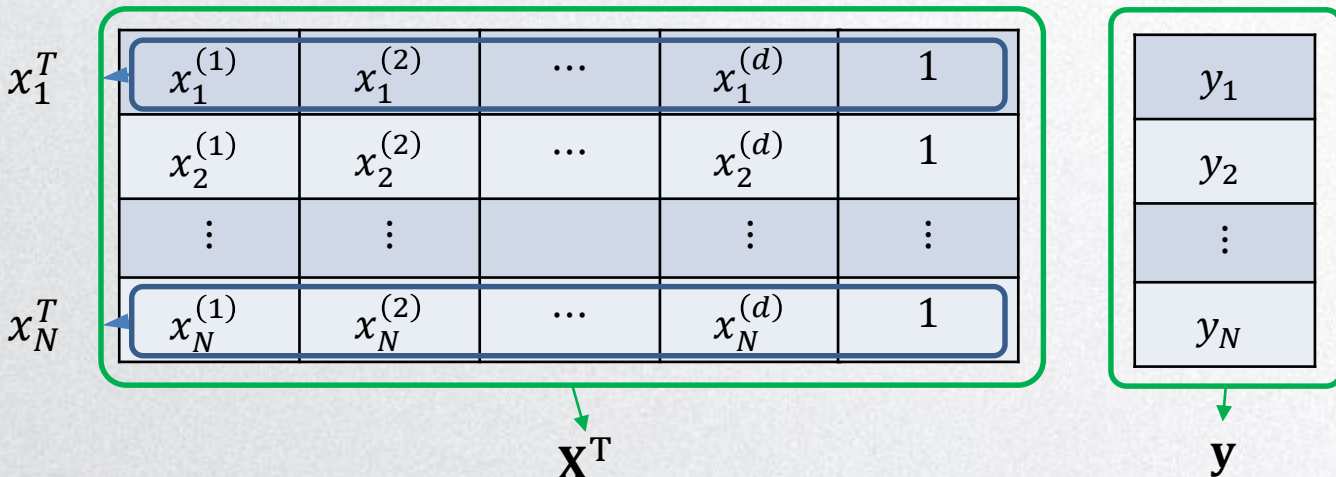
$$\frac{\partial L(\mathbf{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N x_i^{(j)} (p_i - y_i)$$

其中 $p_i = \frac{1}{1+e^{-z_i}} = \frac{1}{1+e^{-(\mathbf{w}^T \mathbf{x}_i)}}$

逻辑回归梯度的具体推导

- Sigmoid函数导数: $p = g(z) = \frac{1}{e^{-z}+1} \Rightarrow \frac{dp}{dz} = \frac{dg}{dz} = g(z)(1 - g(z)) = p(1 - p)$
– $\frac{dp_i}{dw^j} = \frac{dg(\mathbf{w}^T \mathbf{x}_i)}{d\mathbf{w}^j} = g(\mathbf{w}^T \mathbf{x}_i) (1 - g(\mathbf{w}^T \mathbf{x}_i)) \frac{d(\mathbf{w}^T \mathbf{x}_i)}{d\mathbf{w}^j} = p_i(1 - p_i)x_i^j$
- $\frac{\partial L(\mathbf{w})}{\partial w_j} = - \frac{\partial \frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1-y_i) \ln(1-p_i)]}{\partial w_j}$
- $= - \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{p_i} \frac{dp_i}{dw^j} - \frac{1-y_i}{1-p_i} \frac{dp_i}{dw^j} \right]$
- $= - \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i} \right] \frac{dp_i}{dw^j}$
- $= - \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i} \right] p_i(1 - p_i)x_i^j$
- $= - \frac{1}{N} \sum_{i=1}^N [y_i(1 - p_i) - (1 - y_i)p_i]x_i^j$
- $= \frac{1}{N} \sum_{i=1}^N (p_i - y_i)x_i^j$

梯度的矩阵形式



$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \left(\frac{1}{N} \sum_{i=1}^N x_i^{(1)} (p_i - y_i), \frac{1}{N} \sum_{i=1}^N x_i^{(2)} (p_i - y_i), \dots, \frac{1}{N} \sum_{i=1}^N x_i^{(d+1)} (p_i - y_i) \right)$$

- 矩阵形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d + 1)$, $\mathbf{p} - \mathbf{y} = \begin{bmatrix} p_1 - y_1 \\ \vdots \\ p_N - y_N \end{bmatrix}$ 维度为 $N \times 1$



逻辑回归

- 梯度下降

1. 随机初始化参数 w
2. 计算

$$p = \frac{1}{1 + e^{-(w^T x)}}$$

3. 更新参数 w

$$w \leftarrow w - \alpha \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

4. 重复步骤2、3直至收敛



提纲



Python AI
Numpy与科学计算

- □ 一元线性回归
- □ 多元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 逻辑回归
- □ 作业

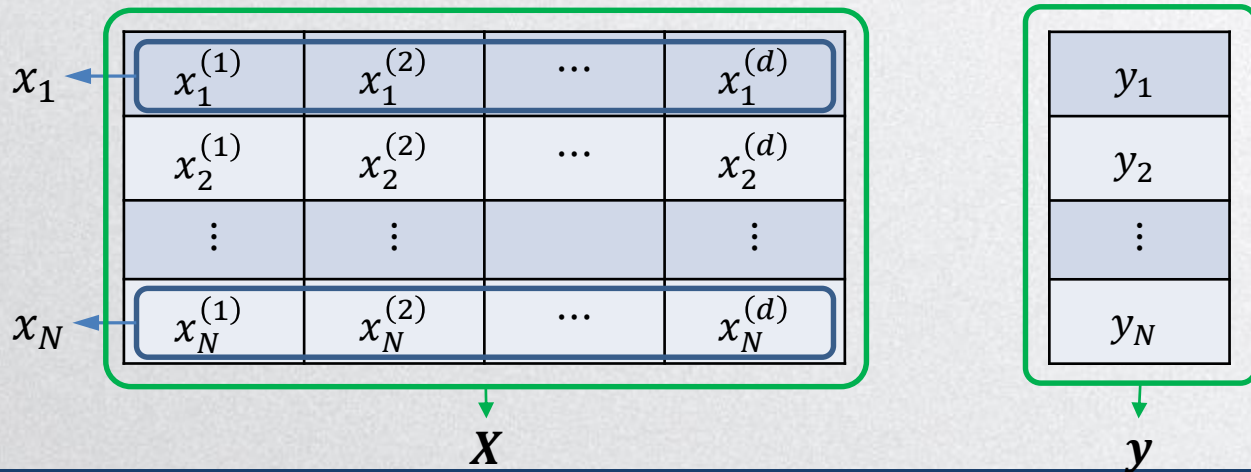


作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 阅读多元线性回归示例代码，理解每个部分的含义
 - 在此基础上进行修改实现逻辑回归代码
 - 注意将计算向量化以提高计算速度，充分利用numpy在矩阵计算上的优势

作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `loss = fit(X,Y)` 函数，用于训练
 - 输入：X为N * d维的训练数据，N为训练样本数，d为数据的维数，即X是N个d维数据点排成的矩阵；Y为N*1维的训练数据真实类别号，即N个类别号排成的向量；
 - 输出：loss为列表，长度为训练轮数，每个元素为对应轮的损失函数值。





作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `loss = fit(X,Y)` 函数，用于训练
 - 实现步骤：
 1. 扩充 x_i ，增加常数1在末尾，并将所有 x_i 组合成 X ，将所有 y_i 组合成 y
 2. 计算损失函数值，加入到loss列表结尾
 3. 计算损失函数对参数 W 的梯度
 4. 更新参数 W
 5. 重复上述过程直至收敛



作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `y_pred, y_pred_label = predict(X)` 函数，用于测试
 - 输入：X为N * d维的测试数据，N为测试样本数；
 - 输出：y_pred维数为N*1，为模型的预测（回归）值，即属于类别1的概率；y_pred_label维数为N*1，为根据回归值得到的预测类别号，只能取0或1
 - y_pred_label的获得方法：若相应的y_pred小于0.5，则y_pred_label为0，否则为1



作业：用numpy实现逻辑回归

- 提交：
 - 把Jupyter Notebook中的空缺部分填好，并运行得到结果
 - 提交文件
- 提交期限：2周
- 提交平台：obe



谢谢！